Today: Introduction to combinatorial computational models

Recap

Ok guys, here is what we have done in Part 1 of the course:

- We saw how to encode objects in binary, and how to define a computational problem
- We saw how to formalize an algorithm: the Turing machine
- We saw how to measure how much time and space an algorithm takes to run
- We also saw non-deterministic Turing machines and how to measure the non-deterministic-time complexity of an algorithm for such a machine.
- This allowed us to define various complexity classes: P, NP, PSPACE, L, NL, E, EXP, etc: a complexity class gathers together all the problems that can be solved with some prescribed amount of computational resources.
- We saw the theory of completeness
    - We saw how to say that "a problem A is not harder than a problem B", which we denoted $A \leq B$.
    - The way we defined this is by the existence of some "simple" function f such that $x \in A \iff f(x) \in B$.
    - By varying what we meant by "simple" (e.g. polytime computable, or logspace computable), we got different notions of reductions
    - This allowed us to define what it means for a problem P to be "hard" for a computational complexity class C. Namely, P is hard for C if every problem $A \in C$ reduces to P.
    - We saw that 3SAT, CLIQUE, etc, are complete for NP (under $\leq_p$)
    - We saw that TQBF is complete for PSPACE (under $\leq_p$)
    - We saw that directed graph reachability is complete for NL (under $\leq_L$)
- We also saw how to prove some lower-bounds, using diagonalization
    - This gave us the hierarchy theorems for time, non-deterministic time, and space
- Finally, we saw the relativization barrier
    - This strongly suggests that diagonalization cannot be used (at least not without some other ingredient) to solve the P vs NP problem!

OK. The above results place us in the beginning of the 1980s, in terms of the development of computational complexity.

The relativization barrier, although a simple result, created a seismic shift in the field. People essentially abandoned diagonalization, and looked elsewhere for how one could answer the P vs NP problem.

Because they had some experience with building actual computers, and used electrical circuits to do so, people's attention turned into more combinatorial ways of defining what computation is.

Boolean circuits

A reasonable abstraction of what the electrical circuits that actually were used in computers did, is the notion of a Boolean circuit.

A *Boolean circuit* is a directed acyclic graph. We think of each edge (or "arc") in the graph as "carrying" a Boolean value, 0 or 1, false or true, and each vertex (or "node") in the graph as computing a function of the values in the incoming edges, the result of which is sent over the outgoing edges. As in the following picture:

(in the board)

Usually we are interested only in cases where the function computed at each node is very simple.

Normally we allow for AND, OR, and NOT functions. This is how we will define circuits for now:

Formal Definition. A **Boolean circuit** over n input bits, is a directed, acyclic graph:

- Each source node (i.e. without incoming edges) is called an "input gate", and is labeled by a number $i \in \{1, \ldots, n\}$, meaning to represent which input $x_1 \ldots x_n$ is chosen.
- Each non-source node having at least two incoming edges is called a "gate", and is labeled by either AND, or OR.
- We can also have non-source nodes with a single incoming edge, and then they must be labelled with NOT.
- We call "wires" to the edges of the graph.
- We call "output gates" to the sinks of the graph.

Given a Boolean circuit C, we usually say that its *size* is the number of wires that it has. Though in the literature sometimes people will use size to mean the number of gates.

The *depth* of a circuit is the length of the longest path in the circuit.

The *fan-in* of a circuit is the maximum number of in-edges at any node.

The *fan-out* of a circuit is the maximum number of out-edges at any node.

Definition. A **Boolean formula** is a Boolean circuit with fan-out 1. I.e. the circuit is actually a tree, and not just a DAG.

Definition. Let C be a Boolean circuit over n input bits with m output gates, which we have ordered in some way. Then C naturally computes a function

$$f : \{0,1\}^n \to \{0,1\}^m$$

as follows.

- For every gate g of the circuit c, we will define a function $f_g : \{0,1\}^n \to \{0,1\}$.
- If g is an input gate, associated with label $i \in [n]$, then $f_g(x_1, \ldots, x_n) = x_i$
- If g is a gate, labeled by OR, and $g_1, \ldots, g_k$ are the gates that have an edge towards g, then

$$f_g(x_1, \ldots, x_n) = \bigvee_{i=1}^{k} f_{g_i}(x_1, \ldots, x_n)$$

- *etc* for AND and NOT

Then the function $f : \{0,1\}^n \to \{0,1\}^m$ computed by the circuit is the function that maps $x_1, \ldots, x_n$ to the sequence of bits computed by the output gates.

Important observation. Here there is a crucial difference with what we have done before. Before, we have defined a single algorithm that should compute a function $f : \{0,1\}^* \to \{0,1\}^*$, i.e., a single algorithm should work for *every* input length!

But here we are only looking at a particular (although generic) input length n.

This difference is important. Before, a single algorithm gave us a pattern that worked for all lengths. Now we forget the pattern. Every input length could be solved by a completely different algorithm.

One calls such models of computation "non-uniform models of computation".

But one can still define notions of complexity quite well.

Definition. Let $f : \{0,1\}^n \to \{0,1\}$. We then define the *size complexity* of f, denoted $S(f)$ to be the smallest size (i.e. fewest number of gates) of any circuit C that computes f on every input $x \in \{0,1\}^n$.

We also define the *depth complexity of* f, $D(f)$, as the smallest depth of any circuit C that computes f on every input.

Finally, we define the *formula size complexity of* f, denoted $L(f)$, as the smallest number of leaves (i.e. source nodes) of any formula that computes f on every input. (Since formulas are trees, counting the number of leaves is the same, up to a constant factor, as counting the number of nodes.)

Examples

The PARITY function: $\oplus_n : \{0,1\}^n \to \{0,1\}$, given by

$$\oplus_n(x_1, \ldots, x_n) = x_1 + \ldots + x_n \mod 2$$

This can be computed by a circuit with $O(n)$ gates, or a formula with n^2 leaves, either having depth $O(\log n)$. Hence: $S(\oplus_n) \leq O(n)$, $L(\oplus_n) \leq O(n^2)$, and $D(\oplus_n) = O(\log n)$

Questions: Can we do better? For circuits no, because of course one needs to read all n inputs to compute the parity function. For formulas: also no, and we will actually see a proof!! (next class)

The *indexing* function $IDX : \{0,1\}^n \times \{0,1\}^k \to \{0,1\}$, defined for $n = 2^k$, given by

$$IDX(x_0, \ldots, x_{n-1}, i) = x_i.$$

This can be computed by a formula of size $O(n \log n)$, and depth 4 (with gates of arbitrary fan-in), or $O(\log n)$ (with gates of fan-in 2). Hence $L(F) = O(n \log n)$

Inner product mod 2: $IP : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$

$$IP(x_1, \ldots, x_n, y_1, \ldots, y_n) = \sum_{i=1}^{n} x_i y_i \mod 2$$

$CLIQUE_{n,k} : \{0,1\}^{\frac{n(n-1)}{2}} \to \{0,1\}$ given by

$$CLIQUE(G) = \begin{cases} 1 & \text{if } G \text{ has a clique of size } k \\ 0 & \text{otherwise} \end{cases}$$

$S(CLIQUE_{n,k}) \leq \binom{n}{k}\binom{k}{2} = O(n^k)$ This is exponential in k... can we do better?

E.g., can we solve $CLIQUE_{n,n/4}$ using circuits of poly(n) size?

Probably not, intuitively, because P != NP, we think.

But what is the relationship between circuits and P?

Basic results on these complexity measures

First observation is that the complexity is bounded:

Theorem. For every single function $F : \{0,1\}^n \to \{0,1\}$, $L(F) = O(2^n)$, $S(F) = O(2^n/n)$, and $D(F) \leq n + \log n + O(1)$.

(can actually show that $L(F) = O(2^n/\log n)$, apparently, but I don't know the proof.)

Proof. To see $L(F) = O(2^n)$, just notice that

$$F(x_1, \ldots, x_n) = 1 \iff (x_1 \wedge F(1, x_2, \ldots, x_n)) \bigvee (\neg x_1 \wedge F(0, x_2, \ldots, x_n))$$

Unpacking this recursively gives us a formula of size $O(2^n)$ that computes F.

To see that $S(F) = O(2^n/n)$, note the following: There are exactly 2^{2^k} different functions on k bits (why?).

So consider the circuit which at the bottom computes all possible k-bit functions on the last k input bits, and at the top expands the first $n-k$ variables, as above. We then connect the functions on the last k bits to the leaves of the top expansion, as necessary. This circuit has size $O(2^{n-k}) + 2^{2^k}O(2^k)$ nodes, which is $O(2^n/n)$ if we choose $k = -2 + \log n$.

Theorem. There exist functions $f : \{0,1\}^n \to \{0,1\}$ such that $S(F) \geq \Omega(2^n/n)$.

Proof. The proof is by counting. We count the number of functions, and the number of circuits of a given size.

The number of n-bit functions is exactly 2^{2^n}. How about the number of circuits over n bit inputs, of size s?

(give me a rough upper-bound.)

Claim. A circuit of size S over n bits can be encoded using $O(S\log(n+S))$ bits. It suffices to encode, for each gate $1, \ldots, S$:

- Whether it is an input gate, or an AND/OR/NOT gate ($\log(n+3)$ bits).

that's number of gates times $\log n$ bits in total, which is $O(S \log n)$ plus counting for each wire:

- The gate it comes from, and the gate it goes to $2 \log S$ bits (because circuit with S wires has fewer than S gates).

So in total we have $O(S(\log(S) + \log n)) = O(S \log S)$ bits.

Hence it follows that, the number of functions computed by circuits of size S is at most the number of circuits of size S which is at most $2^{S \log S}$

Hence, for all functions on n-bits to be computable by circuits of size S, we must have

$$2^{S \log S} \geq 2^{2^n}$$

Which can only happen if $S \geq 2^n/n$. (because otherwise $S \log S \leq 2^n/n$)

Because if $S = o(2^n/n)$ then $S \log S = o(2^n/n \times (n - \log n)) = o(2^n)$

In fact, we have shown something even stronger. If we choose a Boolean function at random, then with overwhelming probability, the function we obtain will have maximimal circuit complexity $2^n/n$.

However:

Tragic fact. We do **not** know how to show, for ANY EXPLICIT FUNCTION, that $S(f) \geq 6n$.

But: we do know how to show, for example, that $L(\oplus_n) \geq n^{2-\varepsilon}$ and we can construct a function f for which we can show a near-cubic lower-bound on $L(f)$. We'll see this next week.

But for formula size, we are stuck at cubic.

On part 2, I plan to do the following:

- Lower-bounds against formula $L(f) >= n^2$.
- Lower-bounds against constant-depth circuits
- mP $\neq$ mNP -- Razborov's famous proof that CLIQUE cannot be computed by monotone polynomial-size circuits.
- And finally, if we have time, we are going to look at pseudorandomness, cryptography, and the natural proofs barrier.